RAG Architecture

Modulo 3 · Lezione 1



Il progetto che avete già in mano

Si riparte esattamente da dove ci eravamo lasciati: il **Progetto Modulo 2**. Un file, quattro funzioni, un toolkit.

```
indicizza_corpus()
cerca_chunk(query, k)
genera_risposta(query, chunks)
estrai_fattura(path)

aikit/ — 7 moduli
```

Ha un nome: **RAG**.

Ha dei difetti: stasera li vediamo **dal vivo**.



Agenda

	Blocco	Dove	Tempo
	Ripasso attivo: sei domande sul Modulo 2	slide	20′
	Il pattern Retriever-Generator	slide	25′
	RAG vs fine-tuning vs long-context	slide	15′
	I quattro failure mode	slide	20′
	Demo: dove fallisce il nostro RAG	`scaffolding/demo_failure.py`	30′
	La mappa del modulo	slide	10′

Stasera nessuno scrive codice: la demo la esegue il docente.



Il formato del ripasso: sei domande

Un mese di pausa: prima di costruire, si riaccende la memoria. Sei domande sul Modulo 2, **a voce**: leggo la domanda, sento due o tre risposte, poi la slide di reveal.

Non è un'interrogazione: è stretching.

`embedding · chunking · similarity · vector DB · structured output · token`



Cos’è un embedding?

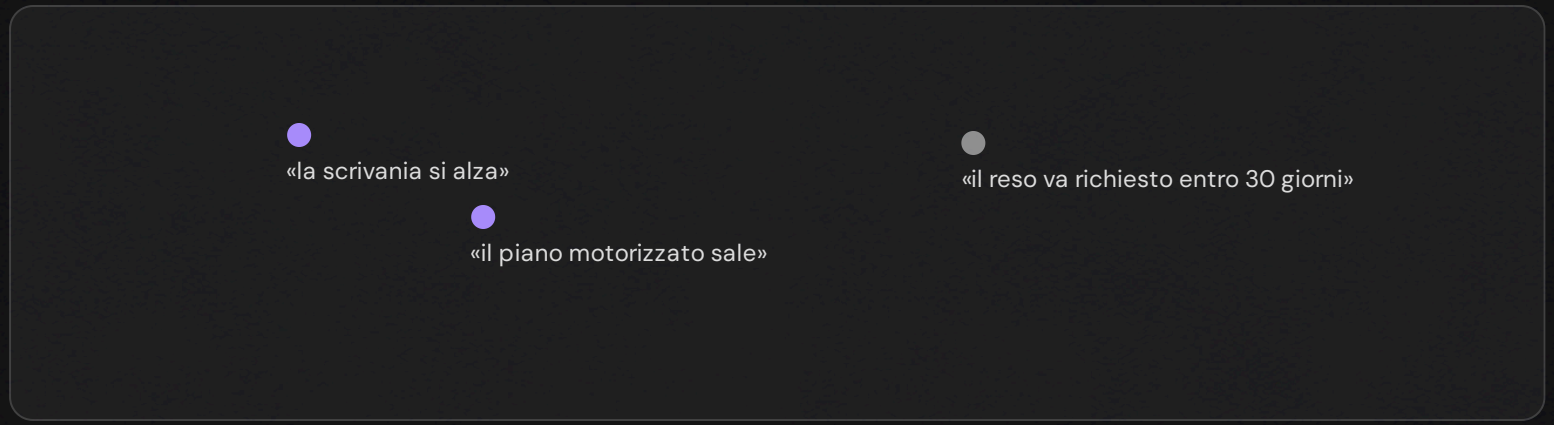
RIPASSO · DOMANDA 1/6



Un embedding è un vettore che rappresenta il significato

RIPASSO · REVEAL 1/6

Testi che dicono cose simili finiscono in **punti vicini** dello spazio. È l'ingrediente che rende possibile *cercare per significato*.



Perché si fa chunking?

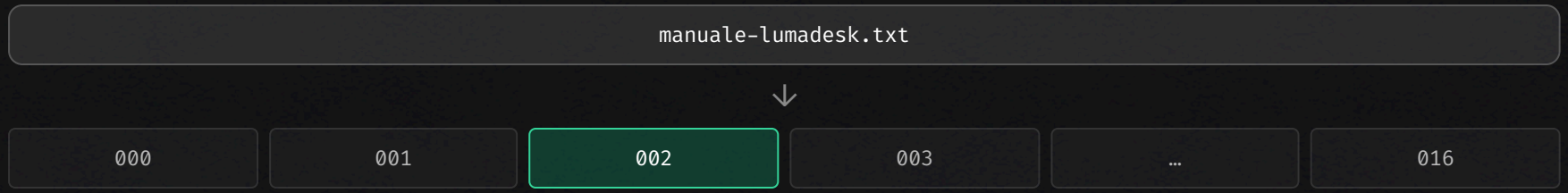
RIPASSO · DOMANDA 2/6



Il chunking definisce l’unità di ricerca

RIPASSO · REVEAL 2/6

Non si cerca tra documenti, si cerca tra **pezzi**: un documento intero *diluisce il segnale* e riempie il contesto di roba inutile. Si spezza per recuperare solo quello che serve.



Solo il chunk che serve entra nel prompt.

Modulo 2 · Lezione 10

Cosa misura la similarity?

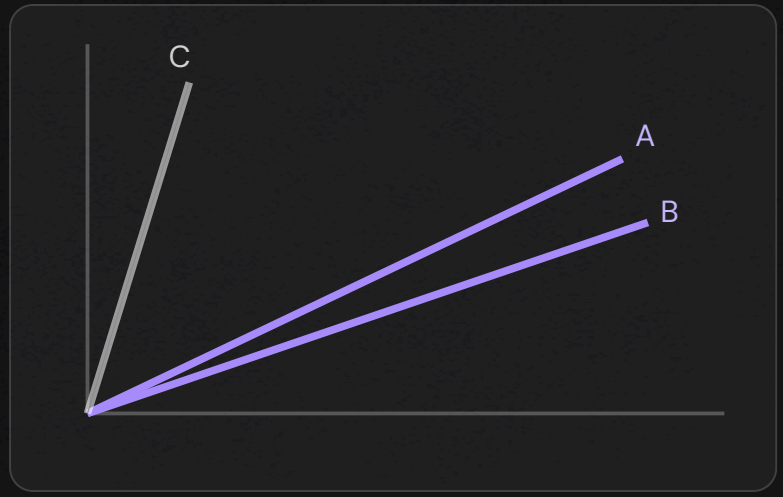
RIPASSO · DOMANDA 3/6



La cosine similarity misura l'angolo tra due vettori

RIPASSO · REVEAL 3/6

Vicino a **1** = stesso significato, vicino a **0** = nessuna relazione. Conta la *direzione*, non la lunghezza.



A ~ B quasi paralleli → ≈ **0.9**

A ~ C quasi ortogonali → ≈ **0.1**

Cosa fa il vector DB?

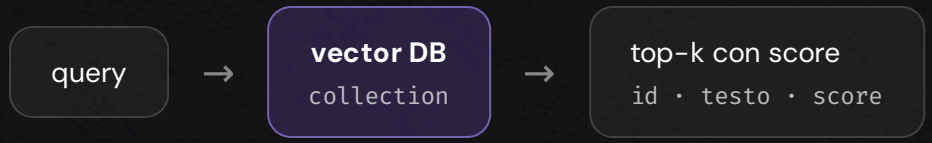
RIPASSO · DOMANDA 4/6



Il vector DB risponde a «i k più simili a questo vettore»

RIPASSO · REVEAL 4/6

Tiene insieme **embedding, testi e metadata**, persistiti su disco, e fa la ricerca *velocemente*. Il nostro: **ChromaDB**, con gli embedding calcolati da noi.



Modulo 2 · Lezione 9



Cosa garantisce lo structured output?

RIPASSO · DOMANDA 5/6



Lo structured output garantisce lo schema, non i valori

RIPASSO · REVEAL 5/6

Campi e tipi della risposta sono quelli dichiarati — **sempre**. Che i valori dentro siano *giusti* resta responsabilità di chi scrive il prompt e di chi verifica.

FORMA GARANTITA

```
risposta: str
fonti: list[str]
trovato: bool
```

VALORI: DA VERIFICARE

Lo schema non sa se la risposta è vera: quello lo controlla il grounding — e stasera lo vedremo fallire.

Modulo 2 · Lezione 13



Per cosa si pagano i token in una chiamata?

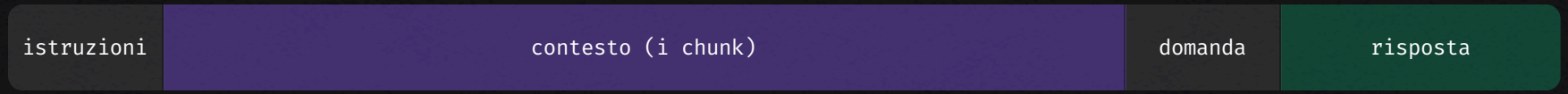
RIPASSO · DOMANDA 6/6



Si paga tutto il prompt più la risposta, e il contesto è la voce grossa

RIPASSO · REVEAL 6/6

Si paga per tutto quello che entra e che esce: **istruzioni + contesto + domanda** in input, **risposta** in output. Nel RAG il contesto è quasi sempre la parte più grande — tenetelo a mente per la demo.



input — si paga per ogni domanda

output

Modulo 2 · Lezione 2

Il vostro progetto era già un RAG

RAG = Retrieval-Augmented Generation: generazione aumentata dal recupero.

INGEST

prepara la conoscenza

`indicizza_corpus()`

RETRIEVE

trova i pezzi giusti

`cerca_chunk()`

GENERATE

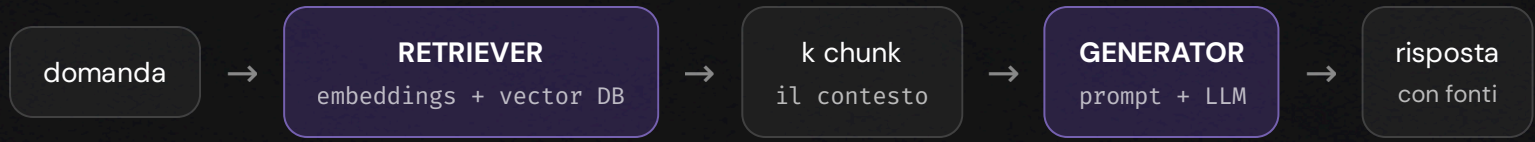
scrive la risposta

`genera_risposta()`

`estrai_fattura()` resta fuori: è estrazione strutturata, non RAG.



Il pattern Retriever-Generator



Il modello **non sa niente** dei vostri documenti: sa solo quello che il retriever gli mette nel prompt, a ogni domanda. Se il retriever sbaglia, il generator non può che sbagliare — o rifiutare.

Il retriever da vicino

Tutti pezzi del Modulo 2: nessuna magia nuova.




```
manuale-lumadesk-013 · score=0.631
```

> Il retriever **restituisce** i chunk, non li nasconde: è la condizione per capire dove si rompe — lo useremo tra poco.

Il generator da vicino

Il prompt assemblato a sezioni — Modulo 2 · Lezione 11, con lo schema di Modulo 2 · Lezione 13.

```
// istruzioni
Rispondi usando SOLO i documenti.
Se non c'è, dillo — senza inventare.

// contesto
<documento fonte="manuale-lumadesk-013">
…</documento>

// domanda
Il piano oscilla: cosa controllo?
```

La risposta strutturata

```
risposta: "Controlla il serraggio…"
fonti: ["manuale-lumadesk-013"]
trovato: true
```

Niente di nuovo — ma adesso ha un **ruolo nel pattern**: è il pezzo che decide cosa fare del contesto.



Grounding e citazioni: a cosa servono

1 · VERIFICABILITÀ

La fonte permette di controllare la risposta sul documento — e di fare **debugging**: tutto il Modulo 3 si regge su questo.

2 · RIFIUTO PULITO

Un RAG che dice «non c'è» vale più di uno che inventa.

3 · FIDUCIA

Nessuno mette in produzione un sistema che risponde «fidati».

Il grounding ha un prezzo — qualche rifiuto di troppo — e stasera lo vedremo.



Indicizzazione offline, interrogazione online

OFFLINE — una volta

load → clean → chunk → embed → index

ONLINE — a ogni domanda

embed query → search → prompt → risposta

> Errori diversi vivono in tempi diversi: un chunking sbagliato lo paghi **a ogni domanda**, ma lo correggi solo **ri-indicizzando**.



I parametri che contano: k, chunk size, soglia

Parametro	Cosa muove	Dove agisce
`k`	quanti chunk entrano nel prompt	online
`chunk size`	quanto è grande l'unità di ricerca	offline (ri-indicizzare)
`soglia`	sotto quale score un chunk resta fuori	online

Oggi nel progetto sono **sparsi nel codice**. Dalla prossima lezione (Modulo 3 · Lezione 2) diventano un punto di configurazione unico: un `CONFIG` solo.



Tre strade per far parlare un modello dei vostri dati

RAG

Recupero i pezzi giusti e li metto nel prompt.

Fine-tuning

Riaddestro i pesi del modello sui miei dati.

Long-context

Metto tutto nel prompt: la finestra è grande.

Nessuna è «quella giusta» in assoluto: si sceglie per criteri.



RAG, fine-tuning, long-context a confronto

Criterio	RAG	Fine-tuning	Long-context
Freschezza dei dati	aggiorni l'indice	ri-addestri	fresco, ma lo ripaghi sempre
Costo per domanda	solo i chunk recuperati	basso	tutto il corpus, ogni volta
Citabilità della fonte	nativa	nessuna	tutto è «la fonte»
Controllo su cosa entra	chunk per chunk	tutto o niente	tutto o niente



Quando basta il long-context

Basta il long-context quando

- corpus **piccolo e stabile**
- **poche** domande
- nessun bisogno di citare

Un contratto da riassumere, non un help desk.

Serve RAG quando

- corpus che **cresce**
- **tante** domande al giorno
- serve la **fonte**

> Il fine-tuning serve per **stile e formato**, non per conoscenza che cambia — e non è materia di questo corso.



I quattro modi in cui un RAG sbaglia

1 · RETRIEVAL MISS

Il chunk giusto non esce dal retriever.

2 · CONTEXT OVERFLOW

Nel prompt entra troppo, e il modello annega.

3 · HALLUCINATION NONOSTANTE IL CONTESTO

La risposta afferma cose che nel contesto non ci sono.

4 · LOST-IN-THE-MIDDLE

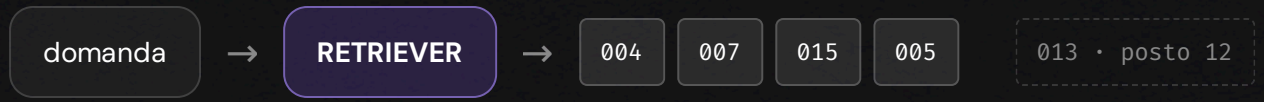
L'informazione c'è, ma sta in mezzo e viene ignorata.

Ora uno per uno: come si riconoscono dall'output.



Retrieval miss

Il chunk giusto non esce dal retriever.



SINTOMO

«Non c'è nei documenti» — ma tu **sai** che c'è. Oppure una risposta costruita su chunk sbagliati.

Causa tipica: utente e documento usano **parole diverse** per la stessa cosa. Si diagnostica guardando i chunk recuperati.

Context overflow

Nel prompt entra troppo, e il modello annega.

k=4 · 737 token input



k=45 · 5.316 token input — ×7



Il chunk giusto ora c'è (posto 12 su 45) — e ogni domanda costa 7 volte tanto.

SINTOMO

Le risposte peggiorano sulle domande **difficili** quando alzi k — e il costo esplode su **tutte**.

IL PIÙ SUBDOLO

Le domande facili continuano a funzionare: non te ne accorgi subito.

Hallucination nonostante il contesto

I chunk sono nel prompt, ma la risposta afferma cose che non ci sono scritte.

IL DOCUMENTO DICE

Come si **attiva** il blocco tastiera: M + «-» per tre secondi.

IL MODELLO RISPONDE

Come si **toglie** — con una procedura mai scritta da nessuna parte.

SINTOMO

Risposta **fluida, plausibile, sicura di sé** — che non passa la verifica sulla fonte. Il sintomo non è «suona male»: è che il controllo sul documento fallisce.

> Il vincolo di grounding (Modulo 2 · Lezione 11) la para **quasi** sempre — non sempre.



Lost-in-the-middle

L'informazione è nel contesto, ma sta in mezzo, e il modello la ignora.



SINTOMO

Stessa domanda, stesso contesto, esiti diversi a seconda di **dove** sta il pezzo giusto: l'ordine dei chunk nel prompt non è un dettaglio.

> I confini non sono muri: l'overflow annega anche perché ciò che conta finisce in mezzo. Quattro nomi, quattro sintomi, **quattro cure diverse**.

Il catalogo dei failure case

Il deliverable della serata: si compila insieme, durante la demo.

#	Domanda	Failure mode	Cosa era stato recuperato	Sintomo
1				
2				
3				
4				

Non si butta: a Modulo 3 · Lezione 4 diventa il **query set** su cui si misura ogni miglioria, a Modulo 3 · Lezione 10 il **golden set** della valutazione formale.



Demo: dove fallisce il nostro RAG

GUARDATE — esegue il docente

`scaffolding/demo_failure.py · 30 minuti`

si rientra dopo la quarta sezione, catalogo compilato

`SEZIONE 1 · retrieval miss`

La domanda che il retriever non capisce.

`SEZIONE 2 · context overflow`

La cura ingenua: alzo k e prendo tutto.

`SEZIONE 3 · hallucination`

La procedura inventata con il contesto nel prompt.

`SEZIONE 4 · lost-in-the-middle`

Stesso contesto, cambia solo la posizione.

Voi guardate chunk e risposte: il catalogo si compila insieme, una riga per failure.



La mappa del modulo: quale lezione cura quale failure

Failure	Cura	Dove
Retrieval miss	hybrid search	Modulo 3 · Lezione 4
	re-ranking	Modulo 3 · Lezione 5
	query transformation	Modulo 3 · Lezione 13
Hallucination	prompt ed evaluation	Modulo 3 · Lezione 10-12
Overflow e lost-in-the-middle	recuperare meglio, mettere nel prompt meno e meglio	tutto il blocco retrieval

Il **come** arriva nelle rispettive lezioni. Tutto confluisce nel Progetto (Modulo 3 · Lezione 15).



La prossima lezione

Si smette di diagnosticare e si ricostruisce: **rag.py**, la baseline unica e configurabile del modulo — le tre funzioni e un `CONFIG` in un posto solo — e il corpus Lumen si allarga a **sei documenti**.

`app.py` → `rag.py` Modulo 3 · Lezione 2

Da lì in poi, ogni lezione cura un pezzo del catalogo di stasera.



Grazie

prossima lezione: Modulo 3 · Lezione 2 — RAG End-to-End

